

ESTIAM · Master 2 · 5ENTAPP / E5WMD

Expense Report Tracker

Application AWS Serverless de gestion de notes de frais

Rapport technique – Architecture, Implémentation & Résultats

AWS Lambda

DynamoDB

API Gateway

Cognito

S3

.NET MAUI

Module

5ENTAPP / E5WMD – Enterprise Software Engineering on AWS

Enseignant

Dr. Abdelhak TOUITI

Formation

Master 2 – ESTIAM Paris

Date

Mai 2026

Dépôt GitHub

Expense_Report_Tracker_GRP-tn-dz

Région AWS

eu-west-3 (Paris)

1

Introduction & Problème industriel

1.1 Contexte

Dans les entreprises modernes, la gestion des notes de frais est un processus critique qui souffre encore, dans de nombreuses organisations, d'un manque de digitalisation : formulaires papier, e-mails non tracés, délais de remboursement longs et risques d'erreur ou de fraude élevés.

1.2 Problème industriel adressé

Ce projet répond à trois enjeux concrets :

- **Traçabilité** : chaque note doit être horodatée, associée à un employé et suivre un cycle de vie précis (création → soumission → approbation/rejet).

- **Contrôle d'accès** : seul un responsable financier (rôle *finance*) peut approuver ou rejeter ; l'employé ne peut agir que sur ses propres notes.
- **Justificatifs numériques** : les reçus doivent être stockés de manière sécurisée et accessibles uniquement via des URLs temporaires.

1.3 Objectifs du projet

Côté technique

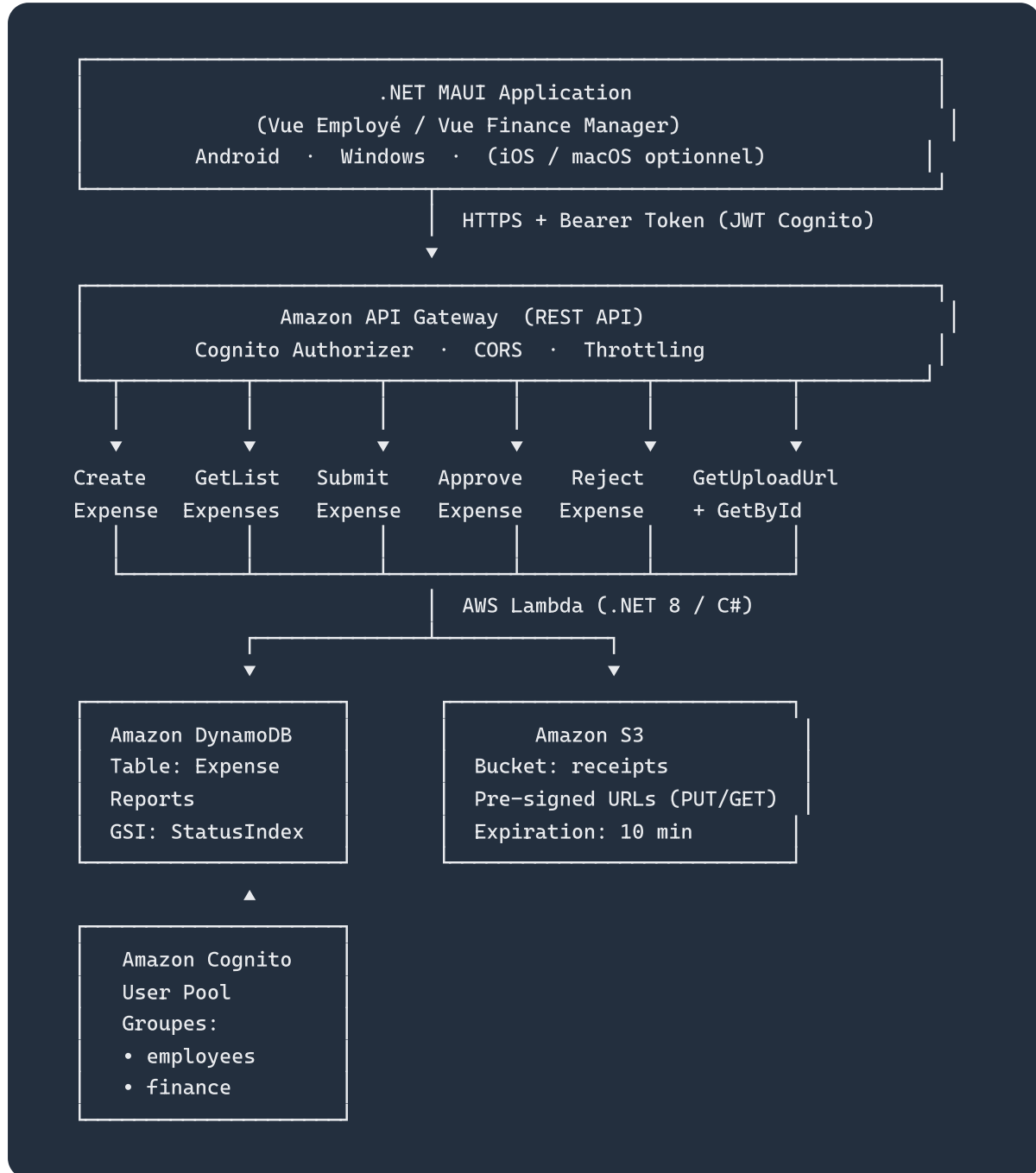
- Architecture 100 % serverless sur AWS
- Authentification JWT via Amazon Cognito
- API REST sécurisée (API Gateway + Cognito Authorizer)
- Frontend multiplateforme (.NET MAUI)

Côté métier

- Cycle de vie DRAFT → SUBMITTED → APPROVED/REJECTED
- RBAC basé sur les groupes Cognito
- Upload de justificatifs via pre-signed URLs S3
- Déploiement continu via AWS CLI

2 Architecture choisie

2.1 Vue d'ensemble – Architecture Serverless AWS



2.2 Justification des choix technologiques

Service AWS	Rôle	Justification
AWS Lambda	Logique métier	Pas de serveur à gérer, paiement à l'usage, scalabilité automatique
API Gateway	Point d'entrée REST	Intégration native Cognito, CORS, throttling, logging CloudWatch
Amazon Cognito	Authentification & RBAC	JWT standards, groupes = rôles, sans gestion de mdp côté app
DynamoDB	Stockage NoSQL	Single-table design, latence < 10 ms, GSI pour les access patterns managers
Amazon S3	Stockage reçus	URLs pré-signées = jamais d'exposition directe du bucket, durée de vie limitée
IAM Role	Sécurité	Least-privilege : Lambda accède uniquement à DynamoDB + S3, pas d'accès console

2.3 Endpoints API déployés

Base URL : <https://rwfqaety92.execute-api.eu-west-3.amazonaws.com/prod>

Méthode	Chemin	Lambda	Rôle requis
POST	/expenses	CreateExpense	employees
GET	/expenses	GetExpenses	employees / finance
GET	/expenses/{id}	GetExpenseById	employees / finance
PUT	/expenses/{id}/submit	SubmitExpense	employees
PUT	/expenses/{id}/approve	ApproveExpense	finance
PUT	/expenses/{id}/reject	RejectExpense	finance
GET	/expenses/{id}/upload-url	GetUploadUrl	employees

3 Modèle de données DynamoDB

3.1 Approche Single-Table Design

DynamoDB favorise le *single-table design* : une seule table stocke tous les types d'entités. Les clés composites (PK + SK) permettent de modéliser des relations et de répondre à plusieurs access patterns sans jointure ni scan coûteux.

3.2 Structure de la table ExpenseReports

Attribut	Type	Description
PK	String	Partition key — <code>USER#<userId></code> (UUID Cognito sub)
SK	String	Sort key — <code>EXPENSE#<expenseId></code> (UUID v4)
expenseId	String	Identifiant unique de la note de frais
userId	String	Identifiant Cognito de l'employé
userEmail	String	Email de l'employé (issu du claim JWT)
amount	Number	Montant en euros
category	String	Catégorie (Transport, Repas, Hébergement...)
description	String	Description libre de la dépense
status	String	État courant : DRAFT SUBMITTED APPROVED REJECTED
StatusGSI_PK	String	Clé du GSI — <code>STATUS#<status></code>
receiptKey	String	Clé S3 du justificatif (optionnel)
justification	String	Motif de rejet (rempli par le manager)
reviewedBy	String	sub Cognito du manager ayant statué
createdAt	String	ISO 8601 UTC — date de création
updatedAt	String	ISO 8601 UTC — dernière mise à jour

3.3 Index secondaire global (GSI) : StatusIndex

Configuration

- Hash key : StatusGSI_PK
- Range key : SK
- Projection : ALL

Utilité

Permet au Finance Manager de récupérer *toutes* les notes avec un statut donné (ex : STATUS#SUBMITTED) sans scanner la table entière.

3.4 Access Patterns couverts

Cas d'usage	Opération DynamoDB	Clé utilisée
Employé : voir ses notes	Query	PK = USER#<userId> , SK begins_with EXPENSE#
Récupérer une note précise	GetItem	PK = USER#<userId> , SK = EXPENSE#<id>
Manager : notes soumises	Query (GSI)	StatusGSI_PK = STATUS#SUBMITTED
Mettre à jour un statut	UpdateItem	PK + SK + UpdateExpression sur status , StatusGSI_PK
Manager : toutes catégories	Query (GSI)	StatusGSI_PK = STATUS#APPROVED / STATUS#REJECTED

Avantage clé : Aucun *Scan* n'est réalisé dans l'application. Chaque accès est un *Query* ou un *GetItem* ciblé, garantissant des latences constantes (< 5 ms) quelle que soit la taille de la table.

3.5 Ressources S3

Le bucket **expense-tracker-receipts-706922781773** stocke les reçus sous la clé **receipts/{userId}/{expenseId}.jpg**. Le bucket est privé : aucun accès public n'est autorisé. Les fichiers sont accessibles uniquement via des pre-signed URLs PUT générées par Lambda, valables 10 minutes.

4 Code clé – RBAC, State Machine & Pre-signed URL

4.1 Contrôle d'accès RBAC via les claims Cognito

Amazon Cognito injecte les groupes de l'utilisateur dans le token JWT sous le claim `cognito:groups`. API Gateway valide la signature du token et transmet les claims à Lambda via le contexte du requêteur.

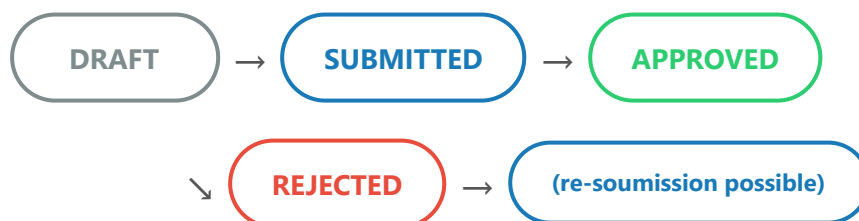
```
// Lecture du claim depuis le contexte API Gateway
var claims = request.RequestContext.Authorizer.Claims;

// Méthode helper partagée dans chaque Lambda
private static bool IsFinanceManager(IDictionary<string, string> claims)
{
    if (claims.TryGetValue("cognito:groups", out var groups))
        return groups.Contains("finance");
    return false;
}

// Exemple d'utilisation dans ApproveExpense
if (!IsFinanceManager(claims))
    return Forbidden("Only Finance Managers can approve expense reports.");
```

L'identifiant de l'utilisateur connecté est extrait du claim `sub` (UUID stable), ce qui évite toute usurpation d'identité : l'employé ne peut accéder qu'à ses propres notes (la PK DynamoDB est construite sur son `sub`).

4.2 Machine à états – Cycle de vie d'une note de frais



Les transitions sont validées **côté serveur** dans chaque Lambda. Le client ne peut pas forcer un statut invalide :

```
// Dans ApproveExpense – validation de la transition
var currentStatus = getResult.Item["status"].S;

if (currentStatus != "SUBMITTED")
{
    return BadRequest(
```

```

        $"Cannot approve an expense with status '{currentStatus}'. " +
        "Only SUBMITTED expenses can be approved.");
    }

    // Mise à jour atomique dans DynamoDB
    await _dynamo.UpdateItemAsync(new UpdateItemRequest
    {
        TableName = _tableName,
        Key = new() {
            ["PK"] = new() { S = $"USER#{body.UserId}" },
            ["SK"] = new() { S = $"EXPENSE#{expenseId}" }
        },
        UpdateExpression = "SET #s = :s, StatusGSI_PK = :gsi, reviewedBy = :mgr, updat
        ExpressionAttributeValues = new() {
            [":s"] = new() { S = "APPROVED" },
            [":gsi"] = new() { S = "STATUS#APPROVED" }
        }
    });

```

4.3 Upload sécurisé via Pre-signed URLs S3

Le reçu n'est jamais transmis via l'API. Lambda génère une URL signée valide 10 minutes, que le client utilise pour un upload direct sur S3 :

```

// GetUploadUrl Lambda - génération de la pre-signed URL
var presignRequest = new GetPreSignedUrlRequest
{
    BucketName = _bucketName,
    Key = $"receipts/{userId}/{expenseId}.jpg",
    Verb = HttpVerb.PUT,
    Expires = DateTime.UtcNow.AddMinutes(10),
    ContentType = "image/jpeg"
};

var uploadUrl = _s3.GetPreSignedURL(presignRequest);

// La clé S3 est sauvegardée dans DynamoDB pour référence future
await _dynamo.UpdateItemAsync( ... "SET receiptKey = :k" ... );

```

Sécurité : Le bucket S3 a le blocage d'accès public activé. Seul Lambda (via son rôle IAM) peut générer des URLs pré-signées. Chaque URL est liée à une clé spécifique et expire après 10 minutes, limitant la fenêtre d'exposition.

4.4 Frontend .NET MAUI – Navigation par rôle

Après l'authentification Cognito, le claim `cognito:groups` est décodé côté client (JWT) pour router vers la bonne vue :

```
// LoginPage.xaml.cs - redirection post-login  
if (result.IsFinanceManager)  
    await Shell.Current.GoToAsync("//ManagerPage");  
else  
    await Shell.Current.GoToAsync("//EmployeePage");
```

5

Conclusion, Difficultés & Améliorations

5.1 Bilan – Ce qui a été réalisé

Composant	Statut	Détail
Infrastructure AWS	✓ Déployé	DynamoDB + S3 + Cognito User Pool + IAM Role + API Gateway
7 Lambda functions	✓ Déployées	C# .NET 8, eu-west-3, rôle IAM minimal
Cognito Authorizer	✓ Configuré	Toutes les routes /prod sauf OPTIONS
Test end-to-end	✓ Validé	Create → Submit → Approve (alice employee, bob finance)
Frontend MAUI	✓ Compilé	Login, EmployeePage, ManagerPage, CreateExpensePage (0 erreur build)
RBAC	✓ Fonctionnel	Rôle employee / finance séparés, 403 si non autorisé
State Machine	✓ Fonctionnel	Validation server-side, transitions invalides bloquées avec message d'erreur
Pre-signed URLs S3	✓ Fonctionnel	Upload reçus, bucket privé, TTL 10 min
Déploiement GitHub	✓ Poussé	27 fichiers, commit feat: deploy all lambdas + complete MAUI frontend

5.2 Difficultés rencontrées

Backend – AWS

- **Conflits de SDK .NET** : le projet utilisait `net8.0` côté Lambda mais `net9.0` côté MAUI. Résolu en maintenant les deux séparément.

Frontend – .NET MAUI

- **SDK mismatch** : Visual Studio 2026 charge .NET 10 SDK par défaut, mais le workload MAUI n'est installé que

- **Cognito Authorizer** : configuration manuelle de chaque route dans API Gateway pour associer l'authorizer tout en excluant les routes OPTIONS (CORS preflight).
- **GSI StatusIndex** : les requêtes des managers nécessitaient de connaître exactement la valeur de clé de partition du GSI (STATUS#SUBMITTED), ce qui a imposé une discipline de nommage stricte à l'écriture.

sur .NET 9.0.203. Résolu via `global.json`.

- **Shell.NavBarTextColor** : attribut déprécié en .NET MAUI 9 → remplacé par `Shell.TitleColor` et `Shell.ForegroundColor`.
- **Packaging Windows** : MAUI Windows nécessite Windows App Runtime (WinUI 3). Installation manuelle requise sur la machine de développement.
- **Format .sln** : `dotnet new sln` génère un format `.slnx` non reconnu par VS. Création manuelle d'un `.sln` classique.

5.3 Axes d'amélioration

- **Infrastructure as Code** : remplacer le script bash par AWS CDK ou Terraform pour un déploiement reproductible et versionné.
- **Notifications** : intégrer Amazon SNS ou SES pour notifier l'employé par e-mail lors d'une approbation ou d'un rejet.
- **Pagination** : les requêtes DynamoDB sont actuellement limitées à 1 Mo. Ajouter un mécanisme de pagination avec `LastEvaluatedKey` pour les grands volumes.
- **CI/CD** : pipeline GitHub Actions pour déployer automatiquement les Lambdas sur chaque push main.
- **Observabilité** : AWS X-Ray pour le tracing distribué (API Gateway → Lambda → DynamoDB) et métriques CloudWatch Dashboard.
- **Tests unitaires** : ajouter des tests xUnit pour chaque Lambda avec mocks DynamoDB (Moq + `Amazon.DynamoDBv2.Mock`).
- **Refresh token** : gérer l'expiration du JWT côté MAUI avec rotation automatique du token via Cognito `REFRESH_TOKEN_AUTH`.

5.4 Synthèse

Ce projet démontre la faisabilité d'une application d'entreprise entièrement serverless sur AWS, avec une gestion des rôles robuste (Cognito + RBAC), une persistance NoSQL optimisée (DynamoDB single-table + GSI), un stockage sécurisé des justificatifs (S3 pre-signed URLs) et un frontend multiplateforme (.NET MAUI).

L'ensemble de l'infrastructure a été déployé et testé en conditions réelles (région **eu-west-3 Paris**), validant un flux complet de bout en bout : création de note → soumission par l'employé → approbation par le manager financier.

Dépôt GitHub : `Expense_Report_Tracker_GRP-tn-dz` — commit `7a6c25e` — 27 fichiers, infrastructure complète + frontend MAUI.

6 Monitoring, Sécurité & Résilience AWS

6.1 CloudWatch Dashboard – Vue temps réel

Un dashboard CloudWatch **ExpenseTracker-Overview** a été configuré en région **eu-west-3** pour superviser l'ensemble de l'infrastructure en temps réel. Il regroupe quatre graphiques couvrant les dimensions critiques de l'application :

Widget	Métriques surveillées	Utilité
Lambda Invocations	CreateExpense, GetExpenses, SubmitExpense, ApproveExpense, RejectExpense	Visualiser le volume d'activité par fonction
Lambda Errors	Erreurs sur CreateExpense, ApproveExpense, RejectExpense	Détecter immédiatement les régressions
Lambda Duration	Durée moyenne des 3 fonctions les plus critiques	Surveiller les dégradations de performance
API Gateway Requests	Count total, 4XXError, 5XXError — stage prod	Distinguer erreurs client vs erreurs serveur

6.2 CloudWatch Alarm – Détection d'erreurs automatique

Une alarme **ExpenseTracker-Lambda-Errors** a été configurée avec les paramètres suivants :

Configuration de l'alarme

- Namespace : AWS/Lambda
- Métrique : **Errors**
- Seuil : **≥ 5 erreurs** en 5 minutes
- Évaluation : 1 point de données
- Missing data : treated as OK

États de l'alarme

-  **OK** — moins de 5 erreurs (état normal)
-  **IN ALARM** — incident détecté → notification
-  **Insufficient data** — démarrage / pas d'activité

État observé : Alarme en statut **OK** — aucune erreur Lambda détectée lors des tests de validation end-to-end (flux Create → Submit → Approve).

6.3 DynamoDB – Sauvegarde et résilience

Fonctionnalité	Configuration	Bénéfice
Point-in-Time Recovery (PITR)	✅ Activé — fenêtre 35 jours	Restauration à la seconde près en cas de suppression accidentelle
Capacity mode	On-demand	Pas de provisioning, scaling automatique, coût à l'usage
Chiffrement	AES-256 (AWS managed key)	Données chiffrées au repos par défaut
GSI StatusIndex	Hash: StatusGSI_PK / Range: SK	Accès O(1) pour les managers sans scan de table

6.4 Amazon S3 – Sécurité des reçus

Mesure	Valeur
Accès public	❌ Bloqué (Block Public Access activé)
Politique d'accès	Seul le rôle IAM Lambda peut générer des pre-signed URLs
CORS	PUT/GET/HEAD autorisés — Content-Type image/jpeg — TTL 1h
Clé S3	<code>receipts/{userId}/{expenseId}.jpg</code> — isolation par employé
Expiration URL	10 minutes — fenêtre d'upload limitée

6.5 IAM – Principe du moindre privilège

Le rôle IAM **expense-tracker-lambda-role** attaché à toutes les Lambda ne donne accès qu'aux services strictement nécessaires :

Permissions accordées

- `dynamodb:GetItem`
- `dynamodb:PutItem`
- `dynamodb:UpdateItem`
- `dynamodb:Query`
- `s3:GetObject`, `s3:PutObject`

Permissions refusées (implicitement)

- ❌ Accès à d'autres tables DynamoDB
- ❌ Accès à d'autres buckets S3
- ❌ Accès IAM, EC2, RDS, etc.
- ❌ Console AWS
- ❌ Suppression de données

- logs:CreateLogGroup,
logs:PutLogEvents